

Practice Sheet: Review how indexing changes as we move from Python lists, to pandas objects, to NumPy arrays. Then apply the core NumPy skills used in spatial computing.

Try the questions **without** running the code. Then create a JN.

Python indexing refresher

```
locations = ["Oak", "Pine", "Maple", "Cedar", "Elm"]
```

1. What value is returned by `locations[0]`?
2. What value is returned by `locations[4]`?
3. What values are returned by `locations[1:4]`?
4. Write an indexing expression that returns "Maple".

```
points = [[100, 200], [140, 230], [180, 260], [120, 280]]
```

5. What does `points[2]` return?
6. What does `points[2][1]` return? Explain what each index selects.
7. Write an expression that returns the x-coordinate of the second point. Assume the coordinates are stored as `[x,y]`.

pandas indexing refresher

```
import pandas as pd

df = pd.DataFrame({
    "name": ["Oak", "Pine", "Maple", "Cedar"],
    "ft": [25, 48, 31, 60],
    "zone": ["A", "B", "A", "B"]
})
```

8. What type of object is returned by `df["ft"]`?
9. Write code to select the name and ft columns together.
10. What row is returned by `df.iloc[1]`?
11. Write code using `iloc` to select the first three rows.
12. Write code to select rows where `ft` is greater than 30.
13. What rows are returned by `df.iloc[1:3]`?

Moving to NumPy arrays

```
import numpy as np

points = np.array([
    [100, 200],
    [140, 230],
    [180, 260],
    [120, 280]
])
```

14. What does `points.shape` return, and what do the two numbers mean?
15. What does `points[0]` return?
16. What does `points[0, 1]` return?
17. What does `points[:, 0]` return? Explain the colon.
18. What does `points[:, 1]` return?
19. Write NumPy indexing code that returns the second and third rows.
20. Compare `points[2][1]` with `points[2, 1]`.

Vectorized operations

```
shift = np.array([10, 20])
```

21. Without running the code, determine the result of `points + shift`.
22. Why does this operation not require a Python for loop?

Boolean masks and spatial queries

```
x = points[:, 0]
y = points[:, 1]

inside = (x >= 110) & (x <= 150) & (y >= 220) & (y <= 290)
```

23. Write the Boolean values stored in `inside[0:]`.
24. What does `points[inside]` return?
25. In GIS language, what spatial operation is this Boolean mask approximating?

np.where() and classification

```
elevation = np.array([240, 510, 430, 680])
classes = np.where(elevation >= 500, 1, 0)
```

26. What values are stored in classes[:]?

27. In your own words, explain the three arguments in np.where(condition, value_if_true, value_if_false).

Distance and pandas connection

```
a = np.array([100, 200])
b = np.array([140, 230])
distance = np.sqrt(((a - b) ** 2).sum())
```

28. What does a - b return?

29. Explain why this calculation is an example of vectorized arithmetic.

```
distance = np.sqrt(((a - b) ** 2).sum())
```

```
ft_array = df["ft"].to_numpy()
```

30. What type of object is ft_array?

31. What happened to the pandas index and column label when the Series was converted to NumPy?

Big picture

32. Complete the table by describing what is being selected.

Structure	Expression	What it selects
Python list	locations[2]	
Nested Python list	points_list[2][1]	
pandas DataFrame	df.iloc[2, 1]	
NumPy array	points[2, 1]	
NumPy array	points[:, 0]	

33. In one sentence: what is the main indexing idea that stays the same across Python, pandas, and NumPy?

34. What becomes easier when data are arranged as a 2-D NumPy array?